

Modül 02: R’da Veri Türleri, Veri Yapıları ve Temel Fonksiyonlar

Prof. Dr. Hakan Mehmetcik

2026-09-10

Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler:

- R’daki temel veri türlerini (numeric, character, logical) tanımlayabilecek,
- Basit matematiksel ve mantıksal işlemleri uygulayabilecek,
- Yerleşik fonksiyonların (sum(), mean(), length(), vb.) kullanımını kavrayabilecek,
- Vektör, liste, matris ve data frame gibi basit veri yapıları ile çalışabilecek,
- Bir CSV dosyasını R’a aktarıp inceleyebilecek.

1. Değişkenler ve Veri Türleri

R dilinde, verileri saklamak ve işlemek için değişkenler kullanılır. Değişkenler, belirli bir değeri tutar ve bu değer üzerinde işlemler yapmanıza olanak tanır.

Değişken Tanımlama (<- Operatörü) R’da bir değişken tanımlamak için <- operatörü kullanılır. Bu operatör, sağdaki değeri soldaki değişkene atar.

```
x <- 10    # x değişkenine 10 değerini atar  
y <- "Merhaba"  # y değişkenine bir karakter dizisi atar
```

R'da Veri Türleri

R'da değişkenlerin farklı veri türleri olabilir. R'ın sık kullanılan veri türleri şunlardır:

- **Sayısal (Numeric):** Ondalıklı veya tam sayı olarak saklanan sayılar.

```
a <- 5.7 # Ondalıklı bir sayısal veri
b <- 10 # Tam sayı olarak bir sayısal veri
```

- **Karakter (Character):** Metin veya harf dizisi olarak saklanan veriler.

```
metin <- "R programlama dili" # Karakter veri türünde bir veri
```

⚠ Değişken adı seçerken dikkat

R, bir değişkene `c`, `data`, `sum`, `mean` gibi **yerleşik fonksiyon adları** vermenizi engellemez. `c <- "metin"` yazmak hata üretmez, ama kodunuzu okuyan (ve birkaç hafta sonra sizin de olacağınız) kişi için ciddi bir kafa karışıklığı kaynağıdır. Kural: **değişken adları fonksiyon adlarıyla çakışmasın.**

- **Mantıksal (Logical):** TRUE veya FALSE değerlerini saklayan değişkenler.

```
durum <- TRUE # Logical
```

Matematiksel ve Mantıksal İşlemler — Kısa Hatırlatma

Aritmetik (+ - * / ^ %% %/), karşılaştırma (== != < > <= >=) ve mantıksal (& | !) operatörlerin tamamını 1. oturumda gördünüz. Bu haftanın konusu operatörler değil, **operatörlerin üzerinde çalıştığı veri türleri ve yapıları**.

Unuttuğunuz bir sözdizimi olursa depo kökündeki `R-Hizli-Referans.qmd` belgesine bakın; ders boyunca öğrendiğiniz her şey orada kümülatif olarak toplanıyor.

Tür Zorlaması: R'ın Sessiz Dönüşümü

Bir vektörün **tüm elemanları aynı türde olmak zorundadır**. Peki farklı türleri aynı `c()` içine koyarsanız R ne yapar? Hata vermez — **sessizce hepsini ortak bir türe çevirir**. Buna *tür zorlaması* (İng. *type coercion*) denir.

Bunun neden önemli olduğunu soyut bir örnekle değil, sizin karşınıza çıkacağı biçimiyle görelim. Bir meclisteki parti sandalye sayılarını kaydediyorsunuz, ama bir parti için sayıyı bilmiyorsunuz:

```
sandalyeler <- c(120, 85, "Bilinmiyor", 45)
class(sandalyeler)
```

```
[1] "character"
```

Vektörün tamamı karaktere döndü. Artık 120 sayısı değil, "120" metnidir. Şimdi ortalamasını almayı deneyin:

```
mean(sandalyeler)
```

```
Warning in mean.default(sandalyeler): argument is not numeric or logical:
returning NA
```

```
[1] NA
```

Dikkat: R **hata vermedi**. Bir uyarı basıp NA döndürdü. Uyarıyı görmezseniz — ki uzun bir script'in ortasında kolayca kaçırırsınız — analizinize sessizce bir NA girmiş olur. Bazı fonksiyonlar ise doğrudan durur:

```
sum(sandalyeler)
```

```
Error in `sum()` :
! invalid 'type' (character) of argument
```

Dönüşüm hiyerarşisi şöyledir — R daima **en esnek** türe doğru gider:

```
logical → integer → double → character
(en katı)                               (en esnek)
```

```
class(c(TRUE, FALSE, 1))      # logical + numeric -> numeric
```

```
[1] "numeric"
```

```
class(c(1, 2, "üç"))          # numeric + character -> character
```

```
[1] "character"
```

```
sum(c(TRUE, TRUE, FALSE))      # TRUE = 1, FALSE = 0 -> 2
```

```
[1] 2
```

Son satır bir tuzak değil, bir **araç**: `sum(x > 5)` ifadesi “kaç eleman 5’ten büyük?” sorusunun en kısa cevabıdır.

! Alan pratiğinde bunun karşılığı

Anket, seçim ve gösterge verilerinde sayısal bir sütunun içine "yok", "-", "N/A", "cevapsız" gibi tek bir metin hücresinin sızması yeterlidir: R **tüm sütunu** karakter okur, ortalama alamazsınız ve hatanın kaynağını sütunun 4.000. satırında ararsınız. Bu yüzden bir veri setini içe aktardıktan sonra ilk yapacağımız şey **str()** çağırıp **her sütunun türünü kontrol etmektir**. Bu alışkanlığı bu hafta edinin.

Eksik Veri: NA Bir Değer Değil, Bir Bilgi Yokluğudur

NA sıfır değildir, boşluk da değildir. “**Bu değer bilinmiyor**” demektir. R’ın NA ile ilgili tüm davranışı bu tek cümleden çıkar:

```
5 + NA
```

```
[1] NA
```

Bilmediğiniz bir şeye 5 eklerseniz, sonucu yine bilemezsiniz. Aynı mantık şaşırtıcı görünen şu sonucu da açıklar:

```
NA == NA
```

```
[1] NA
```

TRUE değil, NA döner. Kimliğini bilmediğiniz iki kişinin aynı kişi olup olmadığını bilemezsiniz. Bu yüzden eksik değeri `==` ile arayamazsınız; `is.na()` kullanırsınız:

```
gelir <- c(4200, NA, 7350, 5100, NA)
is.na(gelir)      # hangi elemanlar eksik?
```

```
[1] FALSE  TRUE  FALSE  FALSE  TRUE
```

```
sum(is.na(gelir)) # kaç tane eksik?
```

```
[1] 2
```

```
mean(gelir) # tek bir NA sonucu NA yapar
```

```
[1] NA
```

```
mean(gelir, na.rm = TRUE)
```

```
[1] 5550
```

 **na.rm = TRUE** masum bir teknik ayar değildir

na.rm = TRUE yazdığınızda R'a “eksik gözlemleri **at**, kalanla hesapla” demiş olursunuz. Teknik olarak sorunsuz. Metodolojik olarak sorulması gereken soru şudur: **kim cevap vermedi?**

Bir gelir anketinde “cevap yok” diyen 500 kişi rastgele dağılmış olabilir — ya da sistematik biçimde en yüksek ve en düşük gelir gruplarından geliyor olabilir. İkinci durumda **na.rm = TRUE** ortalamayı düzeltmez, **yanlı** hale getirir ve bunu sessizce yapar.

Eksik verinin rastgele olup olmadığı, örneklem ve temsil sorunlarının bir parçasıdır; 7. oturumda örnekleme konusunda buraya döneceğiz. Şimdilik kuralı şöyle aklınızda tutun: **na.rm = TRUE** bir çözüm değil, **bir varsayımdır**.

Kategorik Veri: factor

Şu ana kadar üç tür gördünüz: sayısal, karakter, mantıksal. Sosyal bilimde en sık kullanılan tür ise dördüncüsüdür: **factor**, yani kategorik değişken.

Rejim türü (demokrasi / melez / otokrasi), eğitim düzeyi, Likert ölçeği (kesinlikle katılmıyorum ... kesinlikle katılıyorum), bölge, parti — bunlar metin gibi görünür ama metin değildir. Aralarında **sonlu bir kategori kümesi** ve çoğu zaman bir **sıra** vardır.

Sırasız faktör

```
rejim <- factor(c("Demokrasi", "Otokrasi", "Melez", "Demokrasi"))  
  
rejim
```

```
[1] Demokrasi Otokrasi Melez Demokrasi  
Levels: Demokrasi Melez Otokrasi
```

```
levels(rejim)
```

```
[1] "Demokrasi" "Melez" "Otokrasi"
```

```
table(rejim)
```

```
rejim  
Demokrasi Melez Otokrasi  
2 1 1
```

`levels()` kategorilerin kümesini verir. `table()` ise frekans sayımı yapar — karakter vektöründe de çalışır, ama faktörde **hiç gözlenmemiş bir kategori bile** sonuçta sıfır olarak görünür. Bu, eksik kategoriyi fark etmenizi sağlar.

Sıralı faktör

Eğitim düzeyi kategorileri arasında bir sıra vardır ve bu sırayı R'a siz söylersiniz:

```
egitim <- factor(  
  c("Lisans", "Lise", "Doktora", "Yüksek Lisans"),  
  levels = c("Lise", "Lisans", "Yüksek Lisans", "Doktora"),  
  ordered = TRUE  
)  
  
egitim
```

```
[1] Lisans Lise Doktora Yüksek Lisans  
Levels: Lise < Lisans < Yüksek Lisans < Doktora
```

```
egitim[1] > egitim[2] # "Lisans" > "Lise"
```

```
[1] TRUE
```

Son satır önemlidir: R, karakter vektöründe yapamayacağı bir karşılaştırmayı burada yapabiliyor — çünkü **siz ona kategorilerin hiyerarşisini tanımladınız**. R'ın bildiği bir sıra değil, sizin tanımladığınız kavramsal sıra.

⚠ Faktörlerin klasik tuzağı: `as.numeric()`

Bir faktörü sayıya çevirmeye kalkarsanız R size **kategorinin değerini değil, seviye numarasını** verir:

```
puanlar <- factor(c("10", "25", "40"))
as.numeric(puanlar) # 1 2 3 - YANLIŞ!
```

```
[1] 1 2 3
```

```
as.numeric(as.character(puanlar)) # 10 25 40 - doğrusu
```

```
[1] 10 25 40
```

Sayısal bir sütun yanlışlıkla faktör olarak okunmuşsa, doğrudan `as.numeric()` çağırmak veriyi sessizce yok eder. Daima önce `as.character()`.

i Neden şimdi öğreniyoruz?

`factor` üç yerde tekrar karşınıza çıkacak ve her seferinde bu haftaki temele dayanacak: 4. oturumda `ggplot2` kategorileri hangi sırayla çizeceğine faktör seviyelerine bakarak karar verir; 5. oturumda frekans tabloları ve gruplu özetler faktör üzerinden kurulur; 10. oturumda regresyonda kategorik değişkenler **kukla değişkenlere** dönüştürülür ve faktörün ilk seviyesi **referans kategori** olur.

2. R'da Temel Fonksiyonlar

R programlama dilinde temel fonksiyonlar, veri analizi ve istatistiksel hesaplamalar için yaygın olarak kullanılır. İşte bazı temel fonksiyonlar ve örnek kullanımları:

R'nin Yerleşik Fonksiyonları Yerleşik fonksiyonlar, R dilinin varsayılan olarak sunduğu işlevlerdir. Örneğin, `mean()` fonksiyonu bir vektörün ortalamasını hesaplar, `sum()` toplamını verir ve `length()` bir vektörün kaç eleman içerdiğini gösterir.

Örnekler:

1. `print()` Bir nesneyi konsola yazdırır.

```
print("Merhaba Dünya")
```

```
[1] "Merhaba Dünya"
```

2. `sum()`: Bir vektörün elemanlarının toplamını hesaplar.

```
toplam <- sum(c(1, 2, 3, 4, 5))
```

3. `mean()`: Bir vektörün elemanlarının ortalamasını hesaplar.

```
ortalama <- mean(c(1, 2, 3, 4, 5))
```

4. `sd()`: Bir vektörün standart sapmasını hesaplar.

```
standart_sapma <- sd(c(1, 2, 3, 4, 5))
```

5. `length()`: Bir vektörün uzunluğunu döndürür.

```
uzunluk <- length(c(1, 2, 3, 4, 5))
```

6. `seq()`: Belirli bir aralıkta ardışık sayılar oluşturur.

```
dizi <- seq(1, 10, by=2)
```

7. `rep()`: Bir değeri belirli sayıda tekrar eder

```
tekrar <- rep(1, times=5)
```

8. `c()`: Vektör oluşturur.

```
vektor <- c(1, 2, 3, 4, 5)
```

9. `data.frame()`: Veri çerçevesi oluşturur.

```
veri <- data.frame( isim = c("Ali", "Ayşe", "Fatma"), yas = c(25, 30, 35) )
```

10. `summary()`: Bir nesnenin özet istatistiklerini döndürür.

```
ozet <- summary(c(1, 2, 3, 4, 5))
```


Parametrelerle Fonksiyon Kullanımı

Fonksiyonlar, belirli bir işlevi gerçekleştirmek için parametre alabilir. Parametreler, fonksiyonun nasıl çalışacağını belirler. Örneğin, `mean()` fonksiyonuna `na.rm = TRUE` parametresi eklenerek eksik değerleri göz ardı edebilirsiniz.

```
sayilar <- c(10, 20, NA, 40, 50)
ortalama <- mean(sayilar, na.rm = TRUE) # NA değerini göz ardı ederek ortalama hesaplar
```

Fonksiyon Oluşturma

R dilinde kendi fonksiyonlarınızı oluşturmak oldukça basittir. Fonksiyonlar, belirli bir işlevi gerçekleştirmek için bir dizi komut içerir ve gerektiğinde parametreler alabilir. Fonksiyonlar, `function` anahtar kelimesi kullanılarak tanımlanır.

Fonksiyon Oluşturma Bir fonksiyon oluşturmak için aşağıdaki adımları izleyebilirsiniz:

- `function` anahtar kelimesini kullanarak fonksiyonunuzu tanımlayın.
- Fonksiyonun alacağı parametreleri parantez içinde belirtin.
- Fonksiyonun gövdesinde, fonksiyonun gerçekleştireceği işlemleri yazın.
- `return()` fonksiyonunu kullanarak fonksiyonun döndüreceği değeri belirtin (isteğe bağlı).

```
# Toplama fonksiyonu oluşturma
topla <- function(a, b) {
  sonuc <- a + b
  return(sonuc)
}

# Fonksiyonu çağırma
toplama <- topla(5, 3)
print(toplama) # 8
```

```
[1] 8
```

R Fonksiyonları İçin Yardım Alma (? ve help() Komutları)

Bir R fonksiyonu hakkında bilgi almak için `?` veya `help()` komutlarını kullanabilirsiniz. Bu komutlar, ilgili fonksiyonun nasıl kullanılacağı ve parametrelerinin ne olduğu hakkında detaylı bilgi verir.

Örnekler:

```
?mean      # mean() fonksiyonu hakkında yardım alır
help(sum)   # sum() fonksiyonu hakkında yardım alır
```

Bu komutlar, fonksiyonun açıklamasını, hangi argümanları kabul ettiğini ve nasıl çalıştırılacağını gösteren bir dokümantasyon penceresi açar.

Belgeleme ve Fonksiyonların Anlamı

R'ın tüm fonksiyonlarının nasıl çalıştığını, örnekler ve açıklamalarla birlikte görebilirsiniz. R'da her fonksiyonun geniş bir açıklaması bulunur ve bu belgeleme, fonksiyonların nasıl kullanılacağını öğrenmenin etkili bir yoludur.

Örnek:

```
help(mean) # Ortalama hesaplayan mean() fonksiyonunu anlamak için yardım alabiliriz
```

3. Basit Veri Yapıları

R dilinde, veriler çeşitli veri yapıları kullanılarak organize edilir. Temel veri yapıları, vektörler, listeler ve matrislerdir.

Vektörler

R dilinde en temel veri yapılarından biri vektörlerdir. Vektörler, **aynı veri türüne** sahip bir dizi elemanı saklar. Vektör oluşturmak için `c()` fonksiyonu kullanılır.

Bir veri setindeki her sütun aslında bir vektördür. Aşağıda üç ülkenin 2024 demokrasi skorunu tutuyoruz:

```
ulkeler <- c("Türkiye", "Almanya", "Tunus")      # karakter vektör
skorlar <- c(33.4, 89.2, 64.1)                   # sayısal vektör
ab_uyesi <- c(FALSE, TRUE, FALSE)                # mantıksal vektör

# Vektöre isim verme: elemanlar artık konumla değil, adla da anılabilir
names(skorlar) <- ulkeler
skorlar
```

Türkiye	Almanya	Tunus
33.4	89.2	64.1

```
# Elemana erişim: konumla ya da isimle
skorlar[2]
```

```
Almanya
      89.2
```

```
skorlar["Tunus"]
```

```
Tunus
      64.1
```

```
# Elemanı güncelleme
skorlar["Tunus"] <- 65.0

# Vektör üzerinde işlem: tüm elemanlara aynı anda uygulanır
skorlar / 10
```

```
Türkiye Almanya Tunus
      3.34      8.92      6.50
```

```
# Özet
mean(skorlar)
```

```
[1] 62.53333
```

```
length(skorlar)
```

```
[1] 3
```

i Vektörleştirme

`skorlar / 10` yazdığınızda R her elemanı tek tek dolaşmanızı beklemez; işlemi **tüm vektöre birden** uygular. Buna *vektörleştirme* denir ve R'ın diğer dillerden en belirgin farkıdır. Başka dillerde döngü yazacağınız yerlerin çoğunda R'da tek satır yeterlidir.

Mantıksal İndeksleme

Yukarıda vektör elemanlarına **konumlarına** göre eriştik: `vektor[2]` “ikinci elemanı ver” demektir. Ama gerçek veri analizinde soru neredeyse hiç böyle sorulmaz. “Beşinci ülkenin nüfusu nedir?” diye sormayız; “nüfusu 50 milyondan **büyük olan** ülkeler hangileridir?” diye sorarız.

Bunun için R’da köşeli parantezin içine bir konum numarası değil, bir **mantıksal ifade** yazıyoruz.

Adım 1: Karşılaştırma bir mantıksal vektör üretir

```
sayilar <- c(4, 7, 2, 9, 5)
sayilar > 5
```

```
[1] FALSE TRUE FALSE TRUE FALSE
```

Dikkat edin: çıktı bir sayı değil, `sayilar` ile **aynı uzunlukta** bir TRUE/FALSE vektörüdür. Her eleman için “bu koşulu sağlıyor mu?” sorusunun cevabıdır.

Adım 2: Bu maskeyi indeks olarak kullanınız

```
sayilar[sayilar > 5]
```

```
[1] 7 9
```

R, TRUE olan konumlardaki elemanları tutar, FALSE olanları atar. Bu işleme **mantıksal indeksleme** (İng. *logical subsetting*) ya da **maskeleye** denir.

Koşulları 1. oturumda gördüğünüz `&` (ve), `|` (veya), `!` (değil) operatörleriyle birleştirebilirsiniz:

```
sayilar[sayilar > 3 & sayilar < 9] # 3'ten büyük VE 9'dan küçük
```

```
[1] 4 7 5
```

```
sayilar[!(sayilar == 2)] # 2'ye eşit OLMAYANLAR
```

```
[1] 4 7 9 5
```

💡 Sık yapılan hata: = ile == karışıklığı

`sayilar[sayilar = 5]` **hata verir**. Eşitliği sınamak için daima çift eşittir (==) kullanın; tek eşittir (=) atama içindir.

Listeler

Listeler, **farklı türdeki** verileri bir arada saklayabilen veri yapılarıdır. Bir listenin içinde sayı, metin, mantıksal değer ve hatta başka bir vektör bulunabilir.

Listenin tipik kullanımı, tek bir varlığa ait **farklı türden bilgileri** bir arada tutmaktır — bir ülke profili gibi:

```
ulke_profil <- list(
  ulke      = "Tunus",
  yil       = 2011,
  nato_uyesi = FALSE,
  gostergerler = c(enflasyon = 3.5, demokrasi_skoru = 0.65)
)

ulke_profil$ulke
```

```
[1] "Tunus"
```

```
ulke_profil$gostergerler
```

```
      enflasyon demokrasi_skoru
      3.50           0.65
```

```
ulke_profil$gostergerler["enflasyon"] # listenin içindeki vektörün elemanı
```

```
enflasyon
      3.5
```

```
# Eleman güncelleme, ekleme, silme
ulke_profilisi$yil <- 2012
ulke_profilisi$bolge <- "Kuzey Afrika"
ulke_profilisi$nato_uyesi <- NULL

str(ulke_profilisi)
```

List of 4

```
$ ulke      : chr "Tunus"
$ yil       : num 2012
$ gostergerler: Named num [1:2] 3.5 0.65
  ..- attr(*, "names")= chr [1:2] "enflasyon" "demokrasi_skoru"
$ bolge     : chr "Kuzey Afrika"
```

`str()` bir listenin iç yapısını görmenin en hızlı yoludur: kaç eleman var, her biri hangi türde, iç içe geçmiş bir yapı mı.

Matrisler

Matris, satır ve sütunlar hâlinde düzenlenmiş **iki boyutlu ve homojen** bir yapıdır — tüm elemanları aynı türde olmalıdır.

Sosyal bilimde matrisin en doğal karşılığı, satırları ve sütunları **aynı varlık kümesi** olan tablolardır: ülkeler arası ticaret akışları, ittifak ağları, göç akımları. Hücre, satırdaki varlıktan sütundaki varlığa giden miktarı gösterir.

```
# Üç ülke arasındaki ihracat akışı (milyar $)
ticaret <- matrix(
  c( 0, 15, 22,
    14, 0, 8,
    20, 9, 0),
  nrow = 3, byrow = TRUE,
  dimnames = list(
    c("Türkiye", "Almanya", "İtalya"), # satırlar: ihracatçı
    c("Türkiye", "Almanya", "İtalya")  # sütunlar: ithalatçı
  )
)

ticaret
```

	Türkiye	Almanya	İtalya
Türkiye	0	15	22
Almanya	14	0	8
İtalya	20	9	0

```
# Erişim: [satır, sütun]
ticaret["Türkiye", "Almanya"] # Türkiye'den Almanya'ya ihracat
```

```
[1] 15
```

```
ticaret["Türkiye", ] # Türkiye'nin tüm ihracatı (satır)
```

Türkiye	Almanya	İtalya
0	15	22

```
ticaret[, "Türkiye"] # Türkiye'ye tüm ithalat (sütun)
```

Türkiye	Almanya	İtalya
0	14	20

```
# Satır ve sütun toplamaları
rowSums(ticaret) # her ülkenin toplam ihracatı
```

Türkiye	Almanya	İtalya
37	22	29

```
colSums(ticaret) # her ülkenin toplam ithalatı
```

Türkiye	Almanya	İtalya
34	24	30

i byrow = TRUE neden yazdık?

`matrix()` sayıları varsayılan olarak **sütun sütun** doldurur. Yukarıdaki kodu okunabilir kılmak için sayıları satır satır yazdık, bu yüzden R'a “onları yazdığım sırayla, satır satır yerleştir” dedik. `byrow = TRUE` yazmayı unutursanız matris devrik hâlde kurulur ve tüm yorumunuz ters döner — hata mesajı almazsınız, yalnızca yanlış sonuç alırsınız.

Data Frameler

Data frame, sütunları **farklı türde** olabilen iki boyutlu bir yapıdır ve sosyal bilimde çalışacağınız verinin neredeyse tamamı bu biçimdedir: her satır bir gözlem (ülke, kişi, yıl, olay), her sütun bir değişken.

```
panel <- data.frame(  
  ulke      = c("Türkiye", "Almanya", "Tunus", "Polonya", "Fas"),  
  skor      = c(33.4, 89.2, 64.1, 58.7, 41.9),  
  ab_uyesi  = c(FALSE, TRUE, FALSE, TRUE, FALSE),  
  rejim     = factor(c("Melez", "Demokrasi", "Melez", "Demokrasi", "Otokrasi"))  
)  
  
panel
```

ulke	skor	ab_uyesi	rejim
Türkiye	33.4	FALSE	Melez
Almanya	89.2	TRUE	Demokrasi
Tunus	64.1	FALSE	Melez
Polonya	58.7	TRUE	Demokrasi
Fas	41.9	FALSE	Otokrasi

```
str(panel)
```

```
'data.frame':  5 obs. of  4 variables:  
 $ ulke      : chr  "Türkiye" "Almanya" "Tunus" "Polonya" ...  
 $ skor      : num  33.4 89.2 64.1 58.7 41.9  
 $ ab_uyesi  : logi  FALSE TRUE FALSE TRUE FALSE  
 $ rejim     : Factor w/ 3 levels "Demokrasi","Melez",...: 2 1 2 1 3
```

`str()` çıktısına dikkat edin: `ulke` karakter, `skor` sayısal, `ab_uyesi` mantıksal, `rejim` ise faktör — ve faktörün seviyeleri de görünüyor. Dört farklı tür, tek tabloda.

```
# Erişim: [satır, sütun]  
panel[1, 2]          # birinci satır, ikinci sütun
```

```
[1] 33.4
```



```
panel[2, ] # tek satırlık bir data frame (vektör değil!)
```

	ulke	skor	ab_uyesi	rejim
2	Almanya	89.2	TRUE	Demokrasi

```
panel$skor # bir sütunun tamamı, vektör olarak
```

```
[1] 33.4 89.2 64.1 58.7 41.9
```

```
panel$skor[3] # o sütunun üçüncü elemanı
```

```
[1] 64.1
```

```
# Güncelleme
```

```
panel$skor[3] <- 65.0
```

```
# Mantıksal indekslemeyi satır filtrelemek için kullanma
```

```
# Virgülden ÖNCESİ satırları, SONRASI sütunları seçer:
```

```
panel[panel$skor > 60, ]
```

	ulke	skor	ab_uyesi	rejim
2	Almanya	89.2	TRUE	Demokrasi
3	Tunus	65.0	FALSE	Melez

```
# Yeni sütun ekleme
```

```
panel$nufus_mn <- c(85.3, 83.2, 12.0, 36.8, 37.1)
```

```
# Yeni satır ekleme: sütun adları birebir aynı olmalıdır
```

```
yeni <- data.frame(ulke = "Yunanistan", skor = 76.4, ab_uyesi = TRUE,  
                  rejim = "Demokrasi", nufus_mn = 10.4)
```

```
panel <- rbind(panel, yeni)
```

```
panel
```

ulke	skor	ab_uyesi	rejim	nufus_mn
Türkiye	33.4	FALSE	Melez	85.3
Almanya	89.2	TRUE	Demokrasi	83.2
Tunus	65.0	FALSE	Melez	12.0
Polonya	58.7	TRUE	Demokrasi	36.8
Fas	41.9	FALSE	Otokrasi	37.1
Yunanistan	76.4	TRUE	Demokrasi	10.4

```
nrow(panel); ncol(panel); names(panel)
```

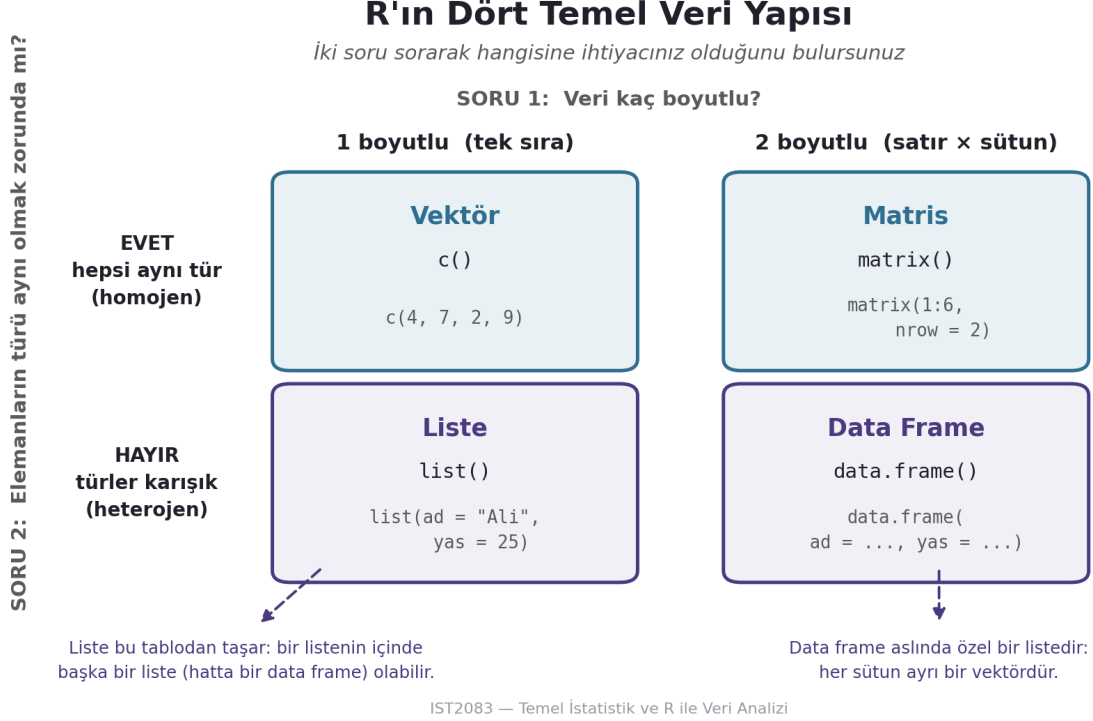
```
[1] 6
```

```
[1] 5
```

```
[1] "ulke"      "skor"      "ab_uyesi"  "rejim"     "nufus_mn"
```

Dördünü Birlikte Görmek

Bu dört yapıyı ezberlemenize gerek yok. Elinizdeki veriye **iki soru** sorarsanız hangisine ihtiyacınız olduğunu kendiniz bulursunuz: veri kaç boyutlu, ve elemanların türü aynı olmak zorunda mı?



Figür 1: R'ın dört temel veri yapısının boyut ve tür-homojenliği eksenlerindeki konumu

Şemanın altındaki iki not önemlidir. **Liste bu tablodan taşar:** bir listenin elemanı yine bir liste olabilir, bu yüzden aslında iki boyutlu bir tabloya tam oturmaz — R'ın en esnek yapısıdır. **Data frame ise aslında özel bir listedir:** her sütunu bir vektör olan, tüm sütunları eşit uzunlukta olmak zorunda olan bir liste. Bu yüzden hem `df$sutun` (liste sözdizimi) hem `df[1, 2]` (tablo sözdizimi) çalışır.

i 3. oturuma köprü

`panel[panel$skor > 60, ]` yazımı doğru ama hantaldır: veri çerçevesinin adını iki kez tekrarlamak zorunda kalırsınız. 3. oturumda `dplyr` paketini öğrendiğinizde aynı işi `filter(panel, skor > 60)` biçiminde yazacaksınız. `dplyr`'ın yaptığı sihir bu değildir — **taban R'da zaten var olan bu mantığı** okunabilir hale getirmesidir. Bu yüzden önce burayı öğreniyoruz.

4. Veri İçe Aktarma ve Kaydetme

Şimdiye kadar veriyi elle yazdık. Gerçek çalışmada veri bir dosyadan, bir paketten ya da bir adresten gelir.

Paket İçinde Gelen Veri

En kolay yol: bazı paketler veri setini içinde taşır. 1. oturumda tanıştığınız `gapminder` bunlardan biri — 142 ülkenin 1952–2007 arası yaşam beklentisi, nüfusu ve kişi başı geliri.

```
library(gapminder)

str(gapminder)
```

```
tibble [1,704 x 6] (S3: tbl_df/tbl/data.frame)
 $ country  : Factor w/ 142 levels "Afghanistan",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ continent: Factor w/ 5 levels "Africa","Americas",...: 3 3 3 3 3 3 3 3 3 3 ...
 $ year     : int [1:1704] 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 ...
 $ lifeExp  : num [1:1704] 28.8 30.3 32 34 36.1 ...
 $ pop      : int [1:1704] 8425333 9240934 10267083 11537966 13079460 14880372 12881816 1386...
 $ gdpPercap: num [1:1704] 779 821 853 836 740 ...
```

`str()` çıktısı bu haftanın konularını tek ekranda topluyor: **country** ve **continent** **faktör**, **year** ve **pop** **tamsayı**, **lifeExp** ve **gdpPercap** **ondalıklı sayı**. Dört tür, bir data frame.

```
# Faktör sütunun kategorileri
levels(gapminder$continent)
```

```
[1] "Africa"    "Americas" "Asia"      "Europe"    "Oceania"
```

```
table(gapminder$continent)
```

Africa	Americas	Asia	Europe	Oceania
624	300	396	360	24

```
# Mantıksal indeksleme: 2007'de yaşam beklentisi 80'in üzerindeki ülkeler
uzun_omur <- gapminder[gapminder$year == 2007 & gapminder$lifeExp > 80, ]
uzun_omur[, c("country", "lifeExp")]
```

country	lifeExp
Australia	81.235
Canada	80.653
France	80.657
Hong Kong, China	82.208
Iceland	81.757
Israel	80.745
Italy	80.546
Japan	82.603
New Zealand	80.204
Norway	80.196
Spain	80.941
Sweden	80.884
Switzerland	81.701

Bu tek satır, bu haftanın tamamını kullanıyor: bir data frame, iki mantıksal koşul, & operatörü, virgülle satır/sütun ayrımı ve sütun seçimi.

Dosyadan Veri: CSV ve Excel

```
library(here)
```

here() starts at /Users/kobain/Dersler/IST2083

```
library(readxl)

# Depo içindeki data/ klasöründen bir Excel dosyası
ornek <- read_excel(here("data", "sample_excel.xlsx"))
head(ornek)
```

Product ID	Product Name	Opening Stock	Purchase/ Stock in	Number of Units Sold
Hand-In- Stock	Cost Price Per Unit (USD)	Cost Price Total (USD)		
-----	-----	-----	-----	-----
-----	-----	-----	-----	-----
P101	Laptop	50 20 10 60 1200 72000	P102 Monitor 40 15 5 50 500 25000	P103 Keyboard 60 25 15 70 50 3500
P104 Headphones 30 10 3 37 100 3700	P105 Smartphone 70 30 20 80 900 72000	P106 Tablet 45 18 8 55 700 38500		

CSV için read.csv() aynı biçimde çalışır:

```
veri <- read.csv(here("data", "example_data.csv"))
```

! **setwd()** komutunu unutun

R öğrenirken karşınıza mutlaka şuna benzer bir satır çıkacak:

```
setwd("C:/Users/Ahmet/Masaüstü/odev")
```

Bu satır **yalnızca Ahmet'in bilgisayarında** çalışır. Ödevinizi gönderdiğinizde, bir yıl sonra kendi yeni bilgisayarınızda açtığınızda, ya da Codespaces'te çalıştırdığınızda kodunuz çöker. Üstelik hata mesajı “dosya bulunamadı” der; asıl sorunun çalışma dizini olduğunu anlamanız zaman alır.

here() bu sorunu kökten çözer: proje kökünü (**.Rproj** dosyasının bulunduğu klasörü) **kendisi bulur** ve yolu oradan kurar.

```
here("data", "example_data.csv")
```

Aynı satır sizin bilgisayarınızda, arkadaşınızın bilgisayarında ve Codespaces'te aynı dosyayı işaret eder. Bu, 1. oturumda konuştuğumuz **tekrarlanabilirlik** ilkesinin en somut karşılığıdır: kodunuz başkasının makinesinde çalışmıyorsa, sonuçlarınız da doğrulanamaz.

i Doğrudan internetten okumak

read.csv() bir internet adresini de kabul eder:

```
veri <- read.csv("https://.../dosya.csv")
```

Kullanışlıdır ama kırılgandır: internet yoksa, adres değişirse veya kaynak siteyi kapatırsa analiziniz çalışmaz. Ciddi bir çalışmada veriyi **bir kez indirip** projenizin **data/** klasörüne koyun; hem tekrar üretilebilir olur hem de veri setinin o günkü hâli kayda geçer.

Veri Kaydetme

```
write.csv(panel, here("data", "panel.csv"), row.names = FALSE)
```

```
library(writexl)  
write_xlsx(panel, here("data", "panel.xlsx"))
```

`row.names = FALSE` demezseniz R, satır numaralarını isimsiz bir ilk sütun olarak dosyaya yazar; o dosyayı tekrar okuduğunuzda gereksiz bir X sütunu görürsünüz.

Veri Yapısını İnceleme

Bir veri setini içe aktardıktan sonra **ilk yapacağınız şey** onu tanımdır:

```
str(gapminder)      # kaç satır, kaç sütun, hangi sütun hangi türde
```

```
tibble [1,704 x 6] (S3: tbl_df/tbl/data.frame)
 $ country   : Factor w/ 142 levels "Afghanistan",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ continent : Factor w/ 5 levels "Africa","Americas",...: 3 3 3 3 3 3 3 3 3 3 ...
 $ year      : int [1:1704] 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 ...
 $ lifeExp   : num [1:1704] 28.8 30.3 32 34 36.1 ...
 $ pop       : int [1:1704] 8425333 9240934 10267083 11537966 13079460 14880372 12881816 1386...
 $ gdpPercap : num [1:1704] 779 821 853 836 740 ...
```

```
summary(gapminder) # sayısal sütunlar için özet, faktörler için frekans
```

country	continent	year	lifeExp
Afghanistan: 12	Africa :624	Min. :1952	Min. :23.60
Albania : 12	Americas:300	1st Qu.:1966	1st Qu.:48.20
Algeria : 12	Asia :396	Median :1980	Median :60.71
Angola : 12	Europe :360	Mean :1980	Mean :59.47
Argentina : 12	Oceania : 24	3rd Qu.:1993	3rd Qu.:70.85
Australia : 12		Max. :2007	Max. :82.60
(Other) :1632			
pop	gdpPercap		
Min. :6.001e+04	Min. : 241.2		
1st Qu.:2.794e+06	1st Qu.: 1202.1		
Median :7.024e+06	Median : 3531.8		
Mean :2.960e+07	Mean : 7215.3		
3rd Qu.:1.959e+07	3rd Qu.: 9325.5		
Max. :1.319e+09	Max. :113523.1		

```
head(gapminder) # ilk satırlar
```

country	continent	year	lifeExp	pop	gdpPercap
Afghanistan	Asia	1952	28.801	8425333	779.4453
Afghanistan	Asia	1957	30.332	9240934	820.8530
Afghanistan	Asia	1962	31.997	10267083	853.1007
Afghanistan	Asia	1967	34.020	11537966	836.1971
Afghanistan	Asia	1972	36.088	13079460	739.9811
Afghanistan	Asia	1977	38.438	14880372	786.1134

```
nrow(gapminder); ncol(gapminder)
```

```
[1] 1704
```

```
[1] 6
```

```
names(gapminder)
```

```
[1] "country" "continent" "year" "lifeExp" "pop" "gdpPercap"
```

Bu beş komut, “veriyi tanımak” dediğimiz şeyin ilk adımıdır. Özellikle `str()` çıktısına bakarken şunu sorun: **sayısal olmasını beklediğim bir sütun karakter olarak mı okunmuş?** Bu haftanın tür zorlaması bölümünde gördüğünüz sorun, gerçek veri setlerinde tam olarak burada yakalanır.

i Not

Ek kaynak: Posit **Base R Cheat Sheet**. Bu haftanın sözdizimi **R-Hızlı-Referans.qmd** belgesine de eklendi.